Fun with Toy Computer

Part 2: Programming in Assembly

Problem 1: Verbose translations

Many *Toy* assembly expressions can be translated directly to corresponding machine instructions, but, especially since the number of Toy Computer instructions is so limited, sometimes a single assembly expression is translated to multiple machine instructions.

Notice that *Toy* assembly allows us to move constants to registers. For example, the following program moves constant byte 0xab to register %a and then outputs the value. When we compile the program and look at the generated machine code, we see that ***mv*** makes use of the *load address* machine operation (even though we do not necessarily interpret 0xab as an address in this context), and each instruction is mapped to a single machine language equivalent.

move_byte.asm

```
.main
mv %a 0xab
.u_hex %a
.line
halt
```

Address	Data	
00	7aab	mv %a 0xab
01	ba80	.u_hex %a
02	b0b0	.line
03	0000	halt

However, notice what happens when we try to store the *two byte word* 0xcdef to a register. This time, the compiled machine code is more complicated.

move_word.asm

```
.main
mv %a 0xcdef
.u_hex %a
.line
halt
```

Address	Data	
00	7fcd	?
01	7e08	?
02	5ffe	?
03	7eef	?
04	4aef	?
05	ba80	.u_hex %a
06	b0b0	.line
07	0000	halt

Explain why we can't simply use a single *load address* machine instruction in this case. Identify the machine instructions used to implement this move and explain what's going on.

Problem 2: More verbose translations

Consider the following assembly program that calculates the bitwise **xor** of two binary numbers. This program can be directly translated to machine code

exclusive.asm
`.main`
`mv %0 0b10010`
`mv %1 0b01010`
`xor %a %0 %1`
`.bin %a`
`.line`
`halt`

Address	Data	
00	7012	`mv %0 0b10010`
01	710a	`mv %1 0b01010`
02	4a01	`xor %a %0 %1`
03	ba30	`.bin %a`
04	b0b0	`.line`
05	0000	`halt`

Notice that toy assembly has an instruction ***or*** that moves the *bitwise disjunction* of the values in two registers to a third register. For example, the following program (i) moves 0b10010 to register %0, (ii) moves 0b01010 to register %1, (iii) moves the bitwise disjunction of these two elements to register %a, and finally (iv) outputs this result in binary. However, *Toy* does not have a circuit in the ALU dedicated to *bitwise disjunction*! When we compile the above assembly, the following machine code is generated.

disjunction.asm
`.main`
`mv %0 0b10010`
`mv %1 0b01010`
`or %a %0 %1`
`.bin %a`
`.line`
`halt`

Address	Data	
00	7012	`mv %0 0b10010`
01	710a	`mv %1 0b01010`
02	**3f01**	`?`
03	**4e01**	`?`
04	**4aef**	`?`
05	ba30	`.bin %a`
06	b0b0	`.line`
07	0000	`halt`

Identify the machine instructions used to implement the bitwise disjunction operation, and explain what is going on.

Problem 3: Assembly programming

In part 1, you wrote machine language programs to:

(a) Output terms in an arithmetic sequence (problem 6)

(b) Draw an image input by the user (problem 9)

(c) Count the number of bits set in a word (problem 10)

Rewrite these programs using toy assembly and then reflect on the following questions:

- Did you find it easier to write programs using assembly or machine code?
- In terms of human readability and abstraction, which language is easier to follow?

Problem 4: Self-modifying programs

From the machine's perspective, there is no distinction between instructions and data in the RAM — everything is simply a sequence of bits stored in a register. This problem explores this idea further with a self-modifying program.

Consider the following assembly program.

```
t_prompt:    .ascii "Input "
t_a:         .ascii "A (zero to exit): "
t_b:         .ascii "B: "
t_op:        .ascii "opcode: "
t_result:    .ascii "Result: "
t_bye:       .ascii "Bye!"

mask:        .data 0x0fff
shift:       .data 12

             .main
loop:        .write [t_prompt]
             .write [t_a]
             .input %0
             jz %0 done
             .write [t_prompt]
             .write [t_b]
             .input %1
             .write [t_prompt]
             .write [t_op]
             .input %2
             ld %3 [instruction]
             and %3 [mask]
             lsh %2 [shift]
             xor %3 %2
             st %3 [instruction]

instruction: add %3 %0 %1
             .write [t_result]
             .den %3
             .line
             .line
             jmp loop

done:        .write [t_bye]
             .line
             halt
```

Notice that the line labeled *instruction* (in bold) adds the values in registers 0 and 1 and moves the result to register 3. Now consider the following example run of the program.

(Continued.)

```
Input A (zero to exit): 40
Input B: 2
Input opcode: 1
Result: 42

Input A (zero to exit): 40
Input B: 2
Input opcode: 2
Result: 38

Input A (zero to exit): 40
Input B: 2
Input opcode: 3
Result: 0

Input A (zero to exit): 40
Input B: 2
Input opcode: 4
Result: 42

Input A (zero to exit): 40
Input B: 2
Input opcode: 5
Result: 160

Input A (zero to exit): 40
Input B: 2
Input opcode: 6
Result: 10

Input A (zero to exit): 0
Bye!
```

Even though the user repeatedly inputs values 40 and 2, the program appears to be performing different operations on these two values — it first *adds* them (as expected), but then *subtracts* them, then performs the *bitwise and* on them, then the *bitwise xor*, then the *left shift* and finally the *right shift*. But these latter instructions don't even appear in the assembly program!

Explain how instructions that don't appear in the assembly program are being executed.

(In general, it's considered bad practice to write self-modifying programs because of the increased complexity that is entailed, but it is an interesting topic to explore!)

Problem 5: A machine code programming editor

Write a program that inputs values from the user until the user inputs zero, treats the values as machine code instructions, and then runs the program input by the user. You can make assumptions, such as the user will input a maximum of 32 instructions.

For example, a simple machine language program that inputs two numbers from the user and then outputs the sum might look like the following.

```
PC: 00   ; set address for the first instruction
00: a010 ; signed integer from user to R[0]
01: a110 ; signed integer from user to R[1]
02: 1201 ; sum of R[0] and R[1] to R[2]
03: b210 ; output value in R[2]
04: b0b0 ; output new line
05: 0000 ; halt the program
```

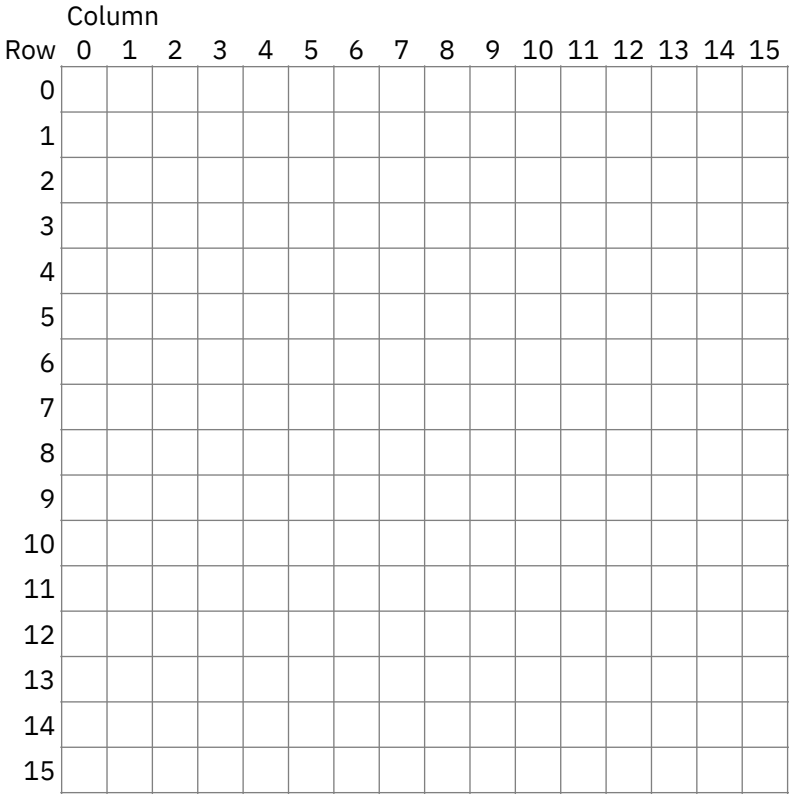
If we input these machine instructions as values into our assembly program, we should get a result similar to the following.

```
Input a program:
0xa010
0xa110
0x1201
0xb210
0xb0b0
0
Program running:
5
10
15
```

(Hint: this problem is very similar to problem 9 of part 1 for which the computer generated an image from data input by the user; in this case, we just need to set the PC to the address where the user's input data starts.)

Problem 6: A computer game

Hurkle is based on an old computer game in which the goal is to find a hurkle hiding in a 16x16 grid. For each turn the user inputs a row and a column to search in and the computer gives the user a vague direction to search for the next guess.



An example run of the game might be something like the following.

```
Welcome to Hurkle!

Guess 1
Row? 7
Column? 7
Go northwest.

Guess 2
Row? 3
Column? 3
Go northwest.

Guess 3
Row? 1
Column? 1
Go east.

Guess 4
Row? 1
Column? 2

You found it!
```

Write an implementation of *Hurkle* in assembly. (Hint: the logic is similar to that used in the example program *guess.asm*.)

Problem 7: Identifying primes

One algorithm that can be used to check whether an integer X is prime is to check whether any of the numbers from X - 1 down to 2 divide X exactly. If so, X is composite (not prime); otherwise X is prime.

An old fashioned BASIC program that implements this algorithm might look something like the following.

```
10 INPUT "WHAT IS YOUR NUMBER"; X
20 IF X <= 1 THEN 130
30 LET T = X
40 T = T - 1
50 IF T = 1 THEN 110
60 LET S = X
70 S = S - T
80 IF S = 0 THEN 130
90 IF S < 0 THEN 40
100 GOTO 70
110 PRINT X; "IS PRIME!"
120 GOTO 140
130 PRINT X; "IS NOT PRIME!"
140 GOTO 10
```

An example run of this BASIC program (using the VintageBasic interpreter):

```
WHAT IS YOUR NUMBER? 1
 1 IS NOT PRIME!
WHAT IS YOUR NUMBER? 2
 2 IS PRIME!
WHAT IS YOUR NUMBER? 3
 3 IS PRIME!
WHAT IS YOUR NUMBER? 4
 4 IS NOT PRIME!
WHAT IS YOUR NUMBER? 5
 5 IS PRIME!
WHAT IS YOUR NUMBER? 6
 6 IS NOT PRIME!
WHAT IS YOUR NUMBER? 7
 7 IS PRIME!
WHAT IS YOUR NUMBER? 8
 8 IS NOT PRIME!
WHAT IS YOUR NUMBER? 9
 9 IS NOT PRIME!
WHAT IS YOUR NUMBER? 10
 10 IS NOT PRIME!
WHAT IS YOUR NUMBER? 11
 11 IS PRIME!
WHAT IS YOUR NUMBER? 27
 27 IS NOT PRIME!
WHAT IS YOUR NUMBER? 29
 29 IS PRIME!
WHAT IS YOUR NUMBER?
```

Interestingly, old fashioned BASIC programs, which rely heavily on GOTO statements, often mirror how we might implement algorithms in assembly.

Write an assembly program that similarly prompts the user for an unsigned integer and then outputs whether the input number is prime.

Problem 8: Working with strings

When a string is assembled using the **.ascii** expression, each pair of characters comprising the string is compressed to fit into a word. For example, if the string “Gandalf” is written to memory starting at an address with label “name”, the following is the result.

Address:	name + 0		name + 1		name + 2		name + 3	
Data:	**4761**		**6e64**		**616c**		**6600**	
Ascii:	47	61	6e	64	61	6c	66	00
Character:	'G'	'a'	'n'	'd'	'a'	'l'	'f'	'\0'
Byte Offset:	0	1	2	3	4	5	6	7

Notice that, given a byte offset b, we can determine the address offset $i = b \text{ div } 2$. For example, for the character at byte offset $b = 3$ (character 'd'), the address offset is $i = 3 \text{ div } 2 = 1$, which means that this character is contained in address `name + 1`.

However, this does not tell us which half of the word at this address contains the character at byte offset b. To determine that, we can calculate $j = b \bmod 2$. If $j = 0$, it means that the character is stored in the left half of the word; otherwise it is stored in the right half. For example, with $b = 3$, $j = 3 \bmod 2 = 1$, which means that the character at byte offset 3 is stored in the right half of the word.

To extract the character at byte offset b, with i and j defined as above, we can then perform bitwise **and** and **shift** operations:

1. Get the value d at address $\texttt{name} + i$.
2. If $j = 1$, then perform bitwise *and* on d and the value 0xff, which gives us the value of the byte in the right half of d. Otherwise, perform bitwise and on d and the value 0xff00, and then right-shift this result by 8 bits, which gives us the value of the byte in the left half of d.

For example, if $d = \texttt{0x6e64}$, then $d \texttt{ \& 0xff} = \texttt{0x64}$, which is the right byte of d and represents the ascii character 'd', and $(d \texttt{ \& 0xff00}) \texttt{ >> 8} = \texttt{0x6e}$, which is the left byte of d and represents the ascii character 'n'.

The only question that remains is, how can we perform $\text{div } 2$ and $\bmod 2$ operations in assembly? It turns out to be pretty easy: if we right-shift a value v by 1 bit, we get $v \text{ div } 2$, and if we perform the bitwise *and* on v and the value 1, we get $v \bmod 2$. (Can you see why?)

Now consider the program *Hello*, a silly game that asks the user to input his or her name and then outputs it backwards ostensibly by mistake. An example run:

```
What is your name? Gandalf
Pleased to meet you, fladnaG!
Whoops! I guess I got it backwards!
Do you prefer it that way (y/n)? y
I knew you would!
Have a nice day!
```

Write an assembly program that implements *Hello*.